

6.2 Embedded Control Hardware

6.2.1 Prototype Motor Control

Author: Sam Schneidereith

Motor control is an important consideration in this project that can easily be overlooked. The resolution and precision of motor control directly impacts the precision of the end effector, which is paramount for docking maneuvers. The six motors attached to the baseplate drive the cable spools which pull the tunnel into the desired position and orientation.

The hardware involved in motor control for the eighth-scale prototype includes the following:

- Arduino Mega 2560 REV3
- Closed Loop Stepper Driver V4.1 for NEMA 17 Stepper Motor
- NEMA 17 Closed Loop Stepper Motor with 20:1 planetary gearbox

Put simply, the Arduino commands the desired motor motion, the stepper drivers convert these commands into electrical signals while supplying sufficient current and voltage, and the motors rotate the commanded number of steps at the specified speed and direction.

The Arduino is programmed in C++ and utilizes the AccelStepper and MultiStepper libraries for simplified simultaneous control of multiple stepper motors.

The NEMA 17 stepper motors used in this project have a native resolution of 200 steps per revolution, corresponding to a step angle of 1.8° per step. However, the integrated 20:1 gearbox increases the effective output resolution by a factor of twenty, yielding

$$200 \times 20 = 4000 \text{ steps per output shaft revolution}$$

This corresponds to an effective angular resolution of

$$\frac{360^\circ}{4000} = 0.09^\circ/\text{step}$$

The motors actuate cable spools with a circumference of approximately 31.1 mm. Therefore, rotational motion must be converted into linear cable displacement. First, the number of output shaft degrees required per meter of cable motion is calculated as

$$\frac{360^\circ}{31.1 \text{ mm}} \times 1000 \text{ mm/m} = 11575.6^\circ/\text{m}$$

Using the gearbox-adjusted angular resolution of 11.11 steps per degree,

$$\frac{4000 \text{ steps}}{360^\circ} = 11.11 \text{ steps/degree}$$

the resulting linear resolution becomes

$$11.11 \times 11575.6 = 1.29 \times 10^5 \text{ steps/m}$$

Thus, the conversion from desired cable displacement to motor steps can be expressed as

$$N_{\text{steps}} = \Delta l \left(\frac{4000}{31.1 \times 10^{-3}} \right)$$

where Δl is the desired cable displacement in meters.

This conversion factor allows desired cable lengths to be directly translated into motor step commands. One important point to note is that this calculation does not account for the thickness of the Kevlar cables meaning that as the cables spool and the spool circumference increases, actual steps per meter will deviate from the calculation above. These deviations can cause length errors on the order of a few millimeters. The avionics team discussed how to work around this and suggestions included parameterizing a length vs degrees curve as the cable spooled, or having the cable run over a rotary encoder down-line of the spool. Cable spooling is not necessarily repeatable, so the latter suggestion would be ideal. In the end, it was decided that due to scope and time constraints, the team was okay with error accumulation on the order of a few millimeters because it didn't affect the qualitative motion of the endplate. Additionally, the primary goal of the prototype was to test the feasibility of this design, so small errors were acceptable.

The control system accepts six cable length values through serial communication via the Arduino's USB type B port. These lengths correspond to the six independently actuated cables of the Stewart-Platform-inspired mechanism. These values are parsed by the Arduino and converted into target motor positions for simultaneous motion using the MultiStepper library. A useful feature of the MultiStepper library is that all motors reach their target step simultaneously by running at individually adjusted rates. This was very useful for avoiding slack in the cables.

Two operational control modes were implemented: manual mode and control mode.

In manual mode, relative cable length changes, Δl , are entered directly through the serial interface. These relative commands incrementally adjust cable lengths and are primarily used for calibration and testing of the system. Since motion is relative, the current cable lengths must continuously be tracked by the controller.

In control mode, an external Python script computes the absolute desired cable lengths using inverse kinematics for a specified end-effector pose or trajectory. The six calculated lengths are

then transmitted to the Arduino through serial communication. Unlike manual mode, these commands represent absolute cable lengths referenced from a calibrated initial configuration. Accurate calibration is therefore critical, as any offset in the initial cable lengths propagates directly into positioning error of the end effector.

After receiving the target lengths, the Arduino converts each desired cable displacement into motor steps and commands all six motors simultaneously to achieve coordinated motion of the tunnel.

The full Arduino source code can be found in Section D.2.

From a controls perspective, this design is quite rudimentary. There was no measurement model, thus the control system was open-loop. If given more time, we had planned to use the endplate-mounted vision system discussed in Section 6.5 to estimate the hatch pose and implement closed-loop control.

6.3 Communication Infrastructure

Author: Sam Schneidereith

Communication between the host computer and the embedded motor controller is achieved through serial communication over USB between a Python-based trajectory generation program and an Arduino Mega 2560 microcontroller. The serial interface operates at a baud rate of 115200 bps, selected to provide sufficiently low latency while maintaining reliable communication.

The Arduino continuously monitors the serial buffer for incoming data packets. Incoming characters are appended to a temporary input buffer until a newline character (`\n`) is detected, indicating the end of a complete command packet. Carriage return characters are ignored to ensure compatibility across operating systems and serial terminal implementations.

The communication protocol utilizes a lightweight ASCII-based packet structure consisting of six comma-separated floating point values corresponding to the desired cable lengths for each actuator:

$$l_1, l_2, l_3, l_4, l_5, l_6$$

where each l_i represents the commanded cable length in meters for actuator i . A complete example packet is shown below:

```
0.5080,0.5079,0.5081, 0.5078,0.5080,0.5079
```

This human-readable packet structure was selected for ease of debugging, transparency during development, and compatibility with standard serial monitoring tools.

After a complete packet is received, the Arduino parses the comma-separated values using tokenization and converts each string element into floating-point numerical data. Basic packet validation is performed by verifying that exactly six values are received before motion commands are executed. Invalid packets generate a “Parse error” response and are discarded.

The Python host computer is responsible for high-level trajectory generation and inverse kinematics calculations. Desired end-effector states are represented as

$$\mathbf{x} = [x, y, z, \phi, \theta, \psi]^T$$

where x , y , and z define translational position and ϕ , θ , and ψ define roll, pitch, and yaw orientation, respectively. For each desired state, the Python controller computes the corresponding Stewart platform cable lengths using inverse kinematics through the `StewartPlatform` class implementation. This class was written by the team and is discussed in detail in Section 6.8.

The resulting six cable lengths are formatted into a serialized ASCII message and transmitted to the Arduino:

```
msg = ",".join(f"{x:.8f}" for x in SP.l_legs) + "\n"
ser.write(msg.encode('ascii'))
```

Upon receiving a valid command packet, the Arduino converts the desired cable lengths into motor step positions and commands all six stepper motors simultaneously using the `MultiStepper` library.

Synchronization between the host computer and embedded controller is achieved using a blocking acknowledgement scheme. After each motion command is executed, the Arduino transmits a completion message:

```
(Arduino) Done
```

The Python controller waits for this acknowledgement before transmitting the next trajectory point. This ensures that sequential trajectory commands are executed in order and prevents command buffer overrun during long-duration actuator movements.

Additional synchronization considerations include suppression of the Arduino automatic reset triggered by serial port initialization. This is accomplished in Python by disabling the DTR signal prior to opening the serial connection:

```
ser.dtr = False
```

Prior to disabling the DTR signal, the Arduino was resetting after each state and re-zeroing at the wrong pose, so the system thought it was somewhere that it was not, which besides losing accurate controllability, this also created the potential for workspace collisions.

mation, a hatch center estimate can be gained from information provided by detection of just one tag. When multiple tags are detected, multiple pose estimates are generated and combined for a fused pose estimate. This multiple-tag strategy allows for greater pose stability and reduces sensitivity to noise or partial occlusion of tags.

6.5.4 Coordinate Frames and Transformations

Author: Sam Schneidereith

Accurate relative pose estimation between the docking tunnel and target hatch requires careful management of multiple coordinate frames and the transformations between them. The vision system implemented in this work detects AprilTags to estimate the six degree-of-freedom relative pose of the hatch with respect to the docking platform (active end of the Stewart Platform). Since measurements originate from cameras mounted on the platform while the desired control quantities are defined relative to the docking geometry, several intermediate coordinate systems must be defined.

The primary coordinate frames used throughout the system are the camera frame, tag frame, hatch frame, and measurement frame.

The **camera frame** is the native coordinate system returned by the AprilTag pose estimation algorithm. Its origin is located at the optical center of the camera, with axes aligned according to the OpenCV camera convention. All raw AprilTag detections are initially expressed in this frame.

The **tag frame** is attached to an individual AprilTag mounted on the docking hatch. Each tag possesses its own local coordinate system centered on the tag itself. Notably, this frame has the z-axis into the tag, with x to the right and y down in the plane of the tag. Knowing the orientation of these axes is particularly important when calculating the Euler angles after a translation or rotation. Since the physical positions and orientations of all tags relative to the hatch are known, each tag can be used to infer the pose of the hatch center.

The **hatch frame** is the primary docking reference frame attached to the center of the passive docking hatch. This frame uses NASA's International Docking System Standard (IDSS) [49] shown in Figure 37. In contrast with the camera frame and tag frame which have the z axis out of the camera and into the tag respectively, this frame has the x axis out of the active object (relevant below) and into the passive object which in this case is the hatch. The origin of this frame is the center of the hatch shown in Figure 36.

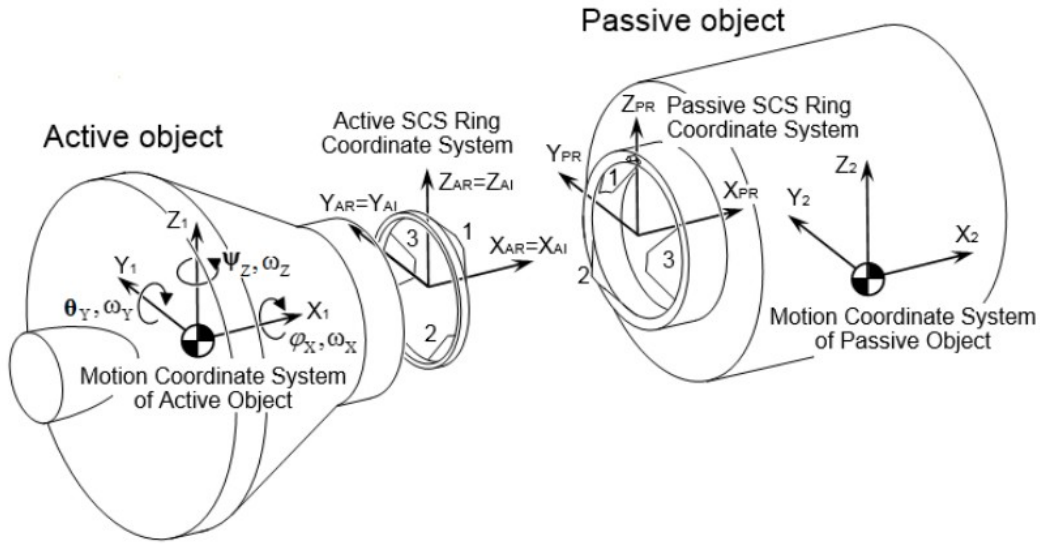


Figure 37 — International Docking System Standard coordinate frames. (NASA, 2026)

Multiple AprilTags mounted around the hatch rim are transformed into this common frame, allowing measurements from different tags to be fused into a single pose estimate.

Finally, the **measurement frame** is a reference frame attached to the center of the active docking platform. This frame exists to support and standardize measurements from multiple cameras simultaneously. Individual camera measurements can be transformed into this shared frame and subsequently fused into a unified estimate. As mentioned above, the camera frame has the z axis out of the camera while the measurement frame, per IDSS, has the x axis out of the surface. Thus a rotation needs to happen, as well as a translation.

Due to time limitations during development, the cameras were not mounted to the tunnel structure, and therefore full multi-camera measurement fusion was not extensively utilized, though it is supported.

Coordinate transformations between frames are represented using homogeneous transformation matrices consisting of both rotation and translation components. A rigid body transformation is written as

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

where

$$R \in \mathbb{R}^{3 \times 3}$$

is a rotation matrix defined as

$$R_{ZYX} = R_Z(\psi)R_Y(\theta)R_X(\phi)$$

and

$$t = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

is a translation vector.

The rotation matrix defines the orientation difference between coordinate frames, while the translation vector defines the positional offset between frame origins. Homogeneous transformations enable both translation and rotation to be represented in a single matrix multiplication operation.

6.5.5 Transforming Points vs. Transforming Frames

A distinction must be made between transforming a point within a frame and transforming the frame itself.

Transforming a **point** changes the coordinates of a physical position while keeping the reference frame fixed. This operation is commonly used when expressing the same physical point in different coordinate systems.

Transforming a **frame** changes the coordinate basis itself. In this case, the physical object remains unchanged while the axes used to describe it are rotated or translated.

Both operations occur throughout the vision pipeline.

The first transformation occurs at the AprilTag level. The AprilTag detector returns the pose of an individual tag relative to the camera frame. However, the control system requires the pose of the hatch center rather than the tag itself. Therefore, the detected tag state is transformed to the hatch center by applying a known geometric offset corresponding to the tag mounting location on the hatch rim. This operation effectively transforms the detected tag pose into the hatch frame.

For a given tag, the transform from the tag frame to the hatch frame is defined as

$$T_{\text{tag} \rightarrow \text{hatch}} = T_{\text{center}} T_{\text{align}}$$

where

$$T_{\text{center}} = \begin{bmatrix} I & t_{\text{tag offset}} \\ 0 & 1 \end{bmatrix}$$

translates the tag origin to the hatch center, and

$$T_{\text{align}} = \begin{bmatrix} R_z(-\theta_{\text{tag}}) & 0 \\ 0 & 1 \end{bmatrix}$$

rotates the tag orientation into alignment with the hatch coordinate system.

The resulting hatch pose in the camera frame is computed as

$$T_{\text{cam} \rightarrow \text{hatch}} = T_{\text{cam} \rightarrow \text{tag}} T_{\text{tag} \rightarrow \text{hatch}} C_{\text{hatch}}$$

where

$$T_{\text{cam} \rightarrow \text{tag}}$$

is the raw measurement of the tag in the camera frame, which can be represented as a transformation from the camera to the tag, and

$$C_{\text{hatch}}$$

is a change-of-basis matrix used to convert between the AprilTag/OpenCV axis convention and the docking hatch convention.

The second major transformation occurs when converting measurements from the camera frame into the platform measurement frame. This operation changes the reference frame used to describe the hatch pose. The camera frame is rotated and translated into the common platform frame using the camera extrinsic calibration matrix.

The camera-to-platform transformation is represented as

$$T_{\text{platform} \rightarrow \text{cam}} = \begin{bmatrix} R_{\text{platform} \rightarrow \text{cam}} & t_{\text{platform} \rightarrow \text{cam}} \\ 0 & 1 \end{bmatrix}$$

A final change-of-basis matrix is then applied to align camera conventions with the docking measurement frame convention:

$$C_{\text{platform}} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

The complete transformation chain used by the system is therefore

$$T_{\text{platform} \rightarrow \text{hatch}} = C_{\text{platform}} T_{\text{platform} \rightarrow \text{cam}} T_{\text{cam} \rightarrow \text{tag}} T_{\text{tag} \rightarrow \text{hatch}} C_{\text{hatch}}$$

This final homogeneous transformation matrix expresses the pose of the docking hatch relative to the platform measurement frame and forms the primary state estimate used for visualization, guidance, and inverse kinematics calculations.

Figure 38 provides a visualization of the frames used. Each apriltag has its coordinate projected, as well as the hatch center coordinate. All 4 of these coordinates are projected while still in the camera frame, then lastly they are transformed into the measurement frame for the final hatch state seen at the top.

pared to a traditional linear actuator based Stewart Platform. This is because the cables driving the platform are only able to exert a tensile force, and the pressure force acts largely normal to the end effector. We believe that the workspace is further limited in roll due to the cables' inability to produce a compressive force.

6.8.2 Inverse Kinematics

Author: Sam Schneidereith

In robotics, the inverse kinematics of a system is the model that derives system inputs for a desired output. For the Stewart platform design, the inverse kinematics model is responsible for converting a desired 6 DoF end-effector pose into the six required cable lengths. Unlike a traditional rigid-link Stewart platform connected by linear actuators, the present system utilizes six independently actuated cables in tension. Consequently, the inverse kinematics problem reduces to determining the Euclidean distance between corresponding base and platform attachment points for a desired platform pose.

The desired end-effector state is represented by the six degree-of-freedom state vector

$$\mathbf{X} = [x, y, z, \phi, \theta, \psi]^T$$

where x , y , and z define translational displacement of the platform center and ϕ , θ , and ψ define roll, pitch, and yaw orientation, respectively.

The orientation of the platform is represented using a rotation matrix constructed from Euler angles:

$$R = R_z(\psi)R_y(\theta)R_x(\phi)$$

where

$$R_x(\phi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \phi & -\sin \phi \\ 0 & \sin \phi & \cos \phi \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix}$$

and

$$R_z(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi & 0 \\ \sin \psi & \cos \psi & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

The complete rigid body transformation from the platform frame to the base frame is then expressed using a homogeneous transformation matrix:

$$T = \begin{bmatrix} R & t \\ 0 & 1 \end{bmatrix}$$

where

$$t = \begin{bmatrix} x \\ y \\ z \end{bmatrix}$$

is the platform translation vector.

The geometric layout of the Stewart platform is defined by six attachment points on both the base and moving platform. These joint locations are represented by

$$B_i = \begin{bmatrix} B_{x,i} \\ B_{y,i} \\ B_{z,i} \end{bmatrix}$$

for the fixed base joints and

$$P_i = \begin{bmatrix} P_{x,i} \\ P_{y,i} \\ P_{z,i} \end{bmatrix}$$

for the moving platform joints.

Because B_i and P_i are both in their local surface frames, the x coordinate for each joint location is 0. Recall from Section 6.5.4 that NASA IDSS has the x axis perpendicular to the surface of the docking ports. For simplicity and consistency, it was decided to keep this convention in the inverse kinematics derivation.

The base and platform joints are distributed in paired configurations around circular mounting rings. The angular spacing between joints is determined from the physical joint spacing and

mounting radius. For the base joints,

$$\alpha_b = \sin^{-1} \left(\frac{s_b/2}{r_b} \right)$$

and similarly for the platform joints,

$$\alpha_p = \sin^{-1} \left(\frac{s_p/2}{r_p} \right)$$

where s_b and s_p are the joint pair spacings and r_b and r_p are the base and platform radii, respectively.

Once the desired pose transformation is defined, each platform joint is transformed into the base coordinate frame according to

$$P_{B,i} = RP_i + t$$

The resulting cable vectors are then computed as

$$L_i = P_{B,i} - B_i$$

and the corresponding cable lengths are obtained from the Euclidean norm:

$$l_i = \|L_i\| = \sqrt{(P_{x,i} - B_{x,i})^2 + (P_{y,i} - B_{y,i})^2 + (P_{z,i} - B_{z,i})^2}$$

for $i = 1, \dots, 6$.

These six cable lengths form the actuator commands transmitted to the motor control system.

6.8.3 Workspace and Reachability Constraints

Unlike conventional Stewart platforms driven by rigid linear actuators, cable-driven systems are subject to additional mechanical constraints arising from the tensile-only nature of cables. Each actuator is capable of producing tension forces but cannot resist compression. As a result, platform stability and controllability depend on maintaining positive tension in all six cables during operation.

6.9 Trajectory Planning

Author: Frank Zhang



Figure 121 — Photo of the 3D printed system

8.3 Inverse Kinematics Validation

Author: Sam Schneidereith

Before the tunnel was attached to the eighth-scale prototype, the avionics team sought to validate the motion control system in isolation. This was accomplished by suspending the hatch beneath the base structure and using the hatch weight as a pseudo-substitute for tunnel pressurization forces opposing cable tension. This configuration can be seen in Figure 122.



Figure 122 — Gravity test configuration (no tunnel)

While implementation of the motor control system was nontrivial, as discussed in Section 6.10, integration with the inverse kinematics model presented in Section 6.8.2 was comparatively

straightforward once reliable synchronized motor actuation had been demonstrated.

Qualitatively, the platform moved as commanded and demonstrated all six degrees of freedom successfully. Motion along the x axis, oriented vertically during the hanging tests, was particularly stable and repeatable. Independent actuation in each translational and rotational degree of freedom was demonstrated, and more complex multi-axis trajectories were also tested successfully.

One complication encountered during gravity testing was reduced controllability during large translations in the yz plane, particularly beyond approximately 15 cm. In these configurations, gravity and cable tension forces were no longer approximately collinear, causing some cables to lose tension and develop slack. The team determined that this limitation was acceptable for preliminary validation because, under nominal operating conditions, tunnel pressurization would provide an opposing force acting normal to the end effector and more closely aligned with the cable directions.

Another limitation of the prototype was the absence of a dedicated measurement or state estimation system. As a result, quantitative validation of end-effector pose accuracy was difficult. The closest approximation achieved involved manually measuring cable lengths using a tape measure and comparing them against the commanded inverse kinematics outputs for a given pose. Measured cable lengths typically differed from commanded values by approximately 1 - 2.5 cm.

These errors were believed to arise primarily from inconsistencies in cable spooling behavior, discussed further in Section 6.2.1. Given that nominal cable lengths were typically on the order of 0.5 m, the resulting error was estimated to be approximately 5%, which was considered acceptable for prototype-level testing and proof-of-concept demonstrations.

Further testing included mounting the tunnel between the vertical assembly to evaluate compressibility and stiffness. The tunnel was compressed to 10 cm demonstrating a stowed configuration. At this point the cables became very taught indicating high tensile loads. While there was never any concern in snapping the Kevlar cables, the 3D printed spools were quickly identified as a mode of failure, having sheared a few during tests (Figure 117b). For fear of damaging the prototype before demonstrations, extreme compression of the tunnel was not explored in depth.



Figure 123 — Tunnel compression test

The softgoods tunnel demonstrated sufficient rigidity to reliably re-expand after compression, even without internal pressurization and with only the platform weight providing restoring force. At the same time, the tunnel remained compliant enough to permit six degree-of-freedom motion after installation, albeit with a somewhat reduced workspace compared to the free hanging configuration.

In particular, large roll maneuvers and translations in the yz plane frequently caused some cables to lose tension and develop slack. It remains inconclusive whether these workspace limitations would persist under nominal pressurized operation, as successful pressure testing was not completed in time for AFSS to perform manipulation experiments with the tunnel under pressure.

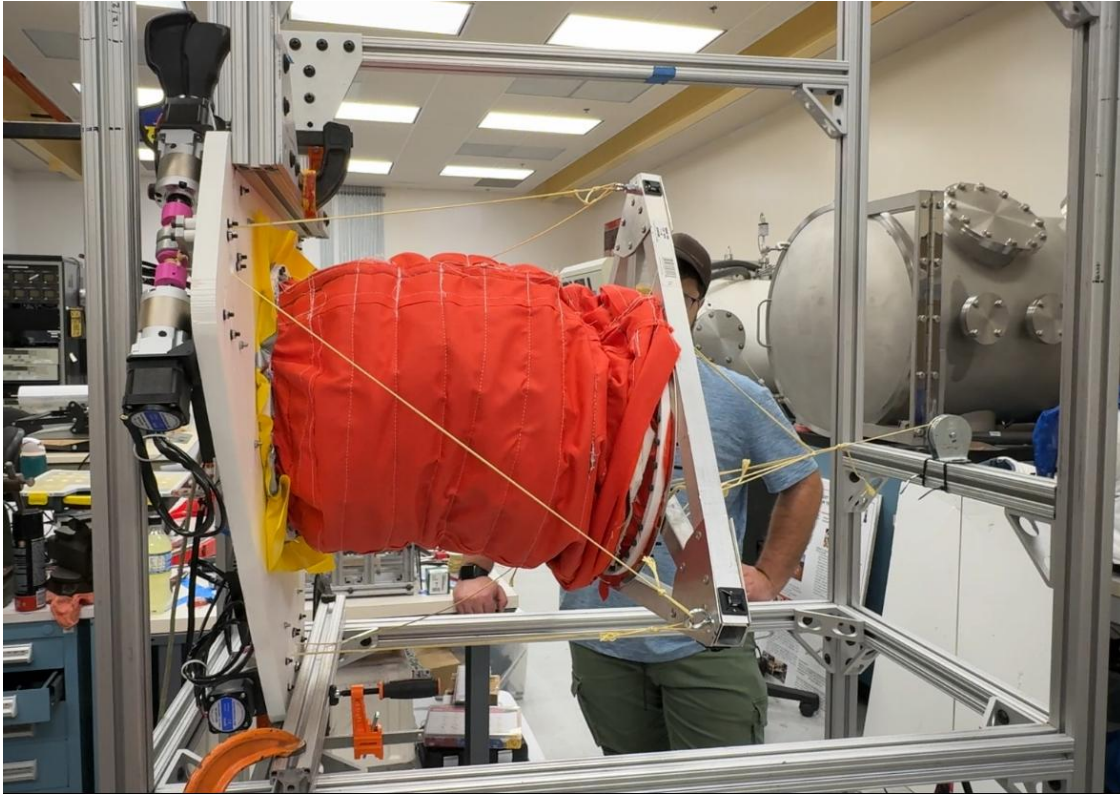


Figure 124 — *Horizontal Test Configuration*

The last major modification to the testing apparatus involved rotating the prototype 90 degrees such that the tunnel was horizontal, and adding a pulley with a weight to provide a substitute for pressurization forces, seen in Figure 124. The primary reason for the modification was for visualization of the tunnel extending and retracting horizontally as intended.

8.4 Vision System Testing

Author: Maanav Subramanian

Validation testing was conducted throughout the lifespan of the project in order to verify the functionality of the vision system. Prior to the arrival of purchased hardware, the code was tested using laptop webcams and printed AprilTags. The code was iteratively tested as it was developed, beginning with initial scripts containing simple detections. This setup just consisted of a laptop webcam and printed AprilTag and ensured the camera recognized the AprilTag by tracing it and displaying its ID.

Once the necessary calculations and frame rotations were implemented, leading to the current working vision system, validation testing was carried out once again. The initial setup for this code involved a laptop webcam and a mock hatch. This mock hatch was constructed as a circle using cardboard. Three AprilTags of 3 different IDs were placed radially, following a setup pic-